

课堂习题

1. 假设系统中有三个任务：T₁, T₂, T₃，其参数如表 1 所示。其中，P 为周期，e 为执行时间，I 为初相（任务第一次开始执行的时刻），D 为相对截止时间。

1) 判断该任务集在 RM、EDF 算法下的可调度性。

2) 用图示方式和简单文字对 LLF 算法情况下各任务执行情况进行说明。（只需给出时间到 44 的情况）

	P	e	I	D
T_1	4	1	0	4
T_2	12	4	2	12
T_3	18	6	5	18

这是一道非常经典的实时系统调度分析题。我们分两步来解答图片中的问题：**可调度性分析**和**LLF调度过程推演**。

第一部分：判断在 RM、EDF 算法下的可调度性

首先，我们需要计算系统的总 CPU 利用率 (U)。

公式为： $U = \sum \frac{e_i}{P_i}$

$$U = \frac{1}{4} + \frac{4}{12} + \frac{6}{18} = \frac{3}{12} + \frac{4}{12} + \frac{4}{12} = \frac{11}{12} \approx 0.9167$$

1. EDF (Earliest Deadline First) 算法可调度性

- **判断依据：** 对于抢占式单处理器系统，不论任务是否有初相（偏移量），只要总 CPU 利用率 $U \leq 1$ ，该任务集在 EDF 算法下就是**可调度的**。
- **结论：** 因为 $0.9167 \leq 1$ ，所以该任务集在 **EDF 算法下可调度**。

2. RM (Rate Monotonic) 算法可调度性

- **充分条件检验：** RM 算法的利用率上限为 $n(2^{1/n} - 1)$ 。当 $n=3$ 时，上限约为 0.779。

由于 $U = 0.9167 > 0.779$ ，无法通过充分条件直接判断，需要进行**响应时间分析 (RTA)**，假设最坏情况（所有任务同时到达，即临界时刻）来求最大响应时间 R_i 。

- **优先级：** RM 根据周期分配优先级，周期越短优先级越高。因此 $T_1 > T_2 > T_3$ 。

- **响应时间计算公式：** $R_i = e_i + \sum_{j \in HP(i)} \lceil \frac{R_i}{P_j} \rceil e_j$

- T_1 : $R_1 = e_1 = 1 \leq D_1(4)$ 。(满足)

- T_2 : $R_2 = 4 + \lceil \frac{R_2}{4} \rceil \times 1$ 。迭代计算得出 $R_2 = 6 \leq D_2(12)$ 。(满足)

- T_3 : $R_3 = 6 + \lceil \frac{R_3}{4} \rceil \times 1 + \lceil \frac{R_3}{12} \rceil \times 4$

- 第1次迭代: $R_3^{(0)} = 6$

- 第2次迭代: $6 + \lceil \frac{6}{4} \rceil \times 1 + \lceil \frac{6}{12} \rceil \times 4 = 6 + 2 + 4 = 12$

- 第3次迭代: $6 + \lceil \frac{12}{4} \rceil \times 1 + \lceil \frac{12}{12} \rceil \times 4 = 6 + 3 + 4 = 13$

- 第4次迭代: $6 + \lceil \frac{13}{4} \rceil \times 1 + \lceil \frac{13}{12} \rceil \times 4 = 6 + 4 + 8 = 18$

- 第5次迭代: $6 + \lceil \frac{18}{4} \rceil \times 1 + \lceil \frac{18}{12} \rceil \times 4 = 6 + 5 + 8 = 19 > D_3(18)$

- **结论：** 由于在最坏情况下 T_3 的响应时间 19 超过了其相对截止时间 18，因此根据 RM 算法的标准判定，该任务集在 **RM 算法下不可调度**。（注：虽然任务有初相，属于异步系统，但标准判定通常以最坏临界情况为准）。

第二部分：LLF (最小松弛度优先) 算法执行情况 (0 到 44)

LLF 核心规则： 任何时刻，优先调度“松弛度”最小的任务。

动态松弛度计算公式: $L = \text{绝对截止时间} - \text{当前时间} - \text{剩余执行时间}$

(绝对截止时间 = 任务到达时间 + 相对截止时间 D)

任务到达时间点：

- **T1:** 0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44...
- **T2:** 2, 14, 26, 38...
- **T3:** 5, 23, 41...

图示化执行甘特图 (Gantt Chart)

时间段	运行任务	关键事件与松弛度 (L) 变化说明
0-1	T1	T1到达。 $L1 = 4 - 0 - 1 = 3$ 。T1执行并在1时刻完成。
1-2	空闲	无任务就绪。
2-4	T2	2时刻T2到达。绝对截止(D)=14。 $L2 = 14 - 2 - 4 = 8$ 。T2执行2个单位。
4-5	T1	4时刻T1到达(D=8)。 $L1 = 8 - 4 - 1 = 3$ 。 T2剩余执行2个单位， $L2 = 14 - 4 - 2 = 8$ 。 $L1 < L2$ ，T1抢占T2。T1在5时刻完成。
5-7	T2	5时刻T3到达(D=23)。 $L3 = 23 - 5 - 6 = 12$ 。 T2剩余2个单位， $L2 = 14 - 5 - 2 = 7$ 。 $L2 < L3$ ，T2恢复执行。T2在7时刻完成。
7-8	T3	只有T3就绪，执行1个单位。
8-9	T1	8时刻T1到达(D=12)。 $L1 = 12 - 8 - 1 = 3$ 。 T3剩余5， $L3 = 23 - 8 - 5 = 10$ 。 $L1 < L3$ ，T1抢占T3并执行完成。
9-12	T3	T3恢复执行，执行3个单位。剩余2。
12-13	T1	12时刻T1到达。 $L1 = 3$ 。T3剩余2， $L3 = 23 - 12 - 2 = 9$ 。T1抢占并执行完成。
13-14	T3	T3恢复执行，执行1个单位。剩余1。
14-15	T3	关键点！ 14时刻T2到达(D=26)。 $L2 = 26 - 14 - 4 = 8$ 。 此时T3剩余1， $L3 = 23 - 14 - 1 = 8$ 。 $L2 = L3$ ，松弛度相同。通常为了减少上下文切换，让当前正在运行的T3继续执行直至完成。
15-16	T2	T2开始执行。
16-17	T1	16时刻T1到达。 $L1 = 3$ 。T2剩余3， $L2 = 26 - 16 - 3 = 7$ 。T1抢占并执行完成。
17-20	T2	T2恢复执行，完成最后3个单位。
20-21	T1	20时刻T1到达。执行并完成。
21-23	空闲	所有就绪任务均已完成，等待下一个任务到来。
23-24	T3	23时刻T3到达(D=41)。 $L3 = 41 - 23 - 6 = 12$ 。执行1个单位。
24-25	T1	24时刻T1到达。抢占T3，执行并完成。
25-26	T3	T3恢复执行1个单位。剩余4。

26-28	T2	26时刻T2到达($D=38$)。 $L_2 = 38 - 26 - 4 = 8$ 。 此时T3剩余4, $L_3 = 41 - 26 - 4 = 11$ 。 $L_2 < L_3$, T2抢占T3。
28-29	T1	28时刻T1到达。 $L_1=3$ 最小。T1抢占执行并完成。
29-31	T2	T2恢复执行, 完成剩余2个单位。
31-32	T3	T3恢复执行。
32-33	T1	32时刻T1到达。 抢占T3, 执行并完成。
33-36	T3	T3恢复执行, 完成最后3个单位。
36-37	T1	36时刻T1到达。 执行并完成。
37-38	空闲	无任务就绪。
38-40	T2	38时刻T2到达($D=50$)。 执行2个单位。
40-41	T1	40时刻T1到达。 抢占T2, 执行并完成。
41-43	T2	41时刻T3到达($D=59$)。 $L_3 = 59 - 41 - 6 = 12$ 。 T2剩余2, $L_2 = 50 - 41 - 2 = 7$ 。 $L_2 < L_3$, T2继续执行并完成。
43-44	T3	T3开始执行至 44 时刻。

(注: 在 $t=14$ 时刻出现了 $L_2 = L_3 = 8$ 的情况, 上文的推演假设了“不抢占正在运行的任务”这一打破平局的常见策略。不同的教材对平局的处理可能略有不同, 但总体逻辑保持一致。)

没问题, LLF (最小松弛度优先) 算法的核心难点在于它是**动态变化**的。也就是说, 随着时间的推移, 任务的优先级一直在变。我们一步一步来拆解这个分析过程。

第一步: 掌握核心“武器”—— 松弛度公式

要分析 LLF, 你只需要牢记一个公式。

松弛度 (Laxity) 直白地说, 就是“一个任务在错过死线之前, 还能‘摸鱼’多长时间”。

- **当前松弛度 = 绝对截止时间 - 当前物理时间 - 任务剩余执行时间**

注意这里的两个关键点:

1. **绝对截止时间 = 任务到达的时刻 + 相对截止时间(D)**。它是一个具体的时间点 (比如“必须在第14秒前交卷”), 而不是一个时间段。
2. **松弛度越小, 越耗不起, 优先级越高**。如果松弛度变成 0, 说明一秒钟都不能等了, 必须马上执行。

第二步: 列出任务的“出场时刻表”

因为题目中带有“初相 (I)”，也就是任务第一次出现的延迟时间，所以我们必须先算出它们分别在哪些时刻到达，以及对应的绝对截止时间。

- **T1 (周期=4, 执行=1, 初相=0, D=4)**
 - 到达时刻: 0, 4, 8, 12, 16, 20...
 - 对应绝对死线: 4, 8, 12, 16, 20, 24...
- **T2 (周期=12, 执行=4, 初相=2, D=12)**
 - 到达时刻: 2, 14, 26, 38...
 - 对应绝对死线: 14, 26, 38, 50...
- **T3 (周期=18, 执行=6, 初相=5, D=18)**
 - 到达时刻: 5, 23, 41...
 - 对应绝对死线: 23, 41, 59...

第三步：像 CPU 一样按时间轴推演（核心分析演示）

我们选取 0 到 15 这个最复杂、最容易出错的时间段，一步步慢动作回放：

【时刻 0】

- 只有 T1 到达系统。
- T1 绝对死线 = 4，当前时间 = 0，需要执行 1。
- $L_1 = 4 - 0 - 1 = 3$ 。
- **动作：** CPU 运行 T1。

【时刻 1】

- T1 运行了 1 个单位，执行完毕。
- 系统里没任务了。
- **动作：** CPU 空闲 (Idle)。

【时刻 2】

- T2 到达（因为它的初相是 2）。
- T2 绝对死线 = 14 (2+12)，当前时间 = 2，需要执行 4。
- $L_2 = 14 - 2 - 4 = 8$ 。
- **动作：** CPU 运行 T2。

【时刻 4】（关键冲突点）

- T1 的第二个周期到了。此时系统里有正在跑的 T2，和刚来的 T1。开始比拼松弛度：
- **算 T1：** 新的绝对死线 = 8 (4+4)，当前时间 = 4，需要执行 1。
 $L_1 = 8 - 4 - 1 = 3$ 。
- **算 T2：** 绝对死线还是 14，当前时间 = 4，它刚才跑了2秒，**剩余执行时间 = 2**。
 $L_2 = 14 - 4 - 2 = 8$ 。
(小技巧：如果一个任务一直在运行，它的松弛度是不会变的，你看刚才L2是8，现在还是8)。
- **比较：** $L_1(3) < L_2(8)$ 。T1 耗不起了！
- **动作：** T1 **抢占** CPU，T2 被赶回去排队。

【时刻 5】（又来新任务了）

- T3 到达（初相是 5）。此时 T1 刚好执行了 1 个单位，跑完了。
- 系统里剩下排队的 T2，和刚来的 T3。比拼松弛度：
- **算 T2：** 绝对死线 14，当前时间 = 5，剩余执行 = 2。
 $L_2 = 14 - 5 - 2 = 7$ 。
(你看，T2 在旁边排队干等了 1 秒，它的松弛度就减少了 1)。
- **算 T3：** 绝对死线 = 23 (5+18)，当前时间 = 5，需要执行 6。
 $L_3 = 23 - 5 - 6 = 12$ 。
- **比较：** $L_2(7) < L_3(12)$ 。
- **动作：** T2 继续运行。

【时刻 7】

- T2 把剩下的 2 个单位跑完了。
- 系统只剩 T3。
- **动作：** T3 开始运行。

【时刻 8】

- T1 第三个周期到达。
- **算 T1：** $L_1 = 12 - 8 - 1 = 3$ 。

- **算 T3:** 死线 23, 当前 8, 刚才跑了1秒, 剩余 5。 $L_3 = 23 - 8 - 5 = 10$ 。
- **动作:** T1 松弛度小, T1 **抢占** T3。

(中间 T1 在时刻 9 跑完, T3 恢复运行, 直到时刻 12 T1 又来抢占一波, T3 再次被打断。这部分逻辑同上, 我们直接跳到下一个最精彩的时刻)

【时刻 14】(平局出现了!)

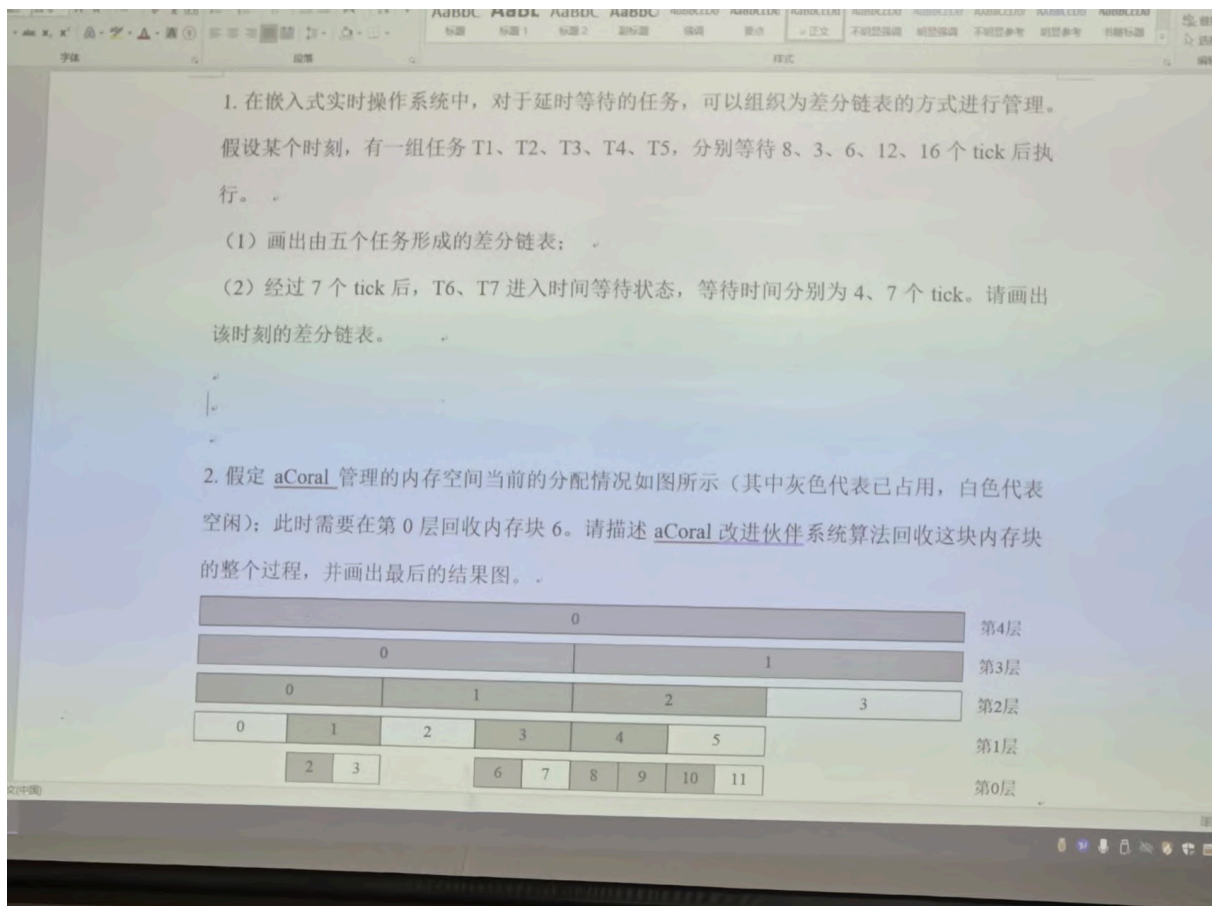
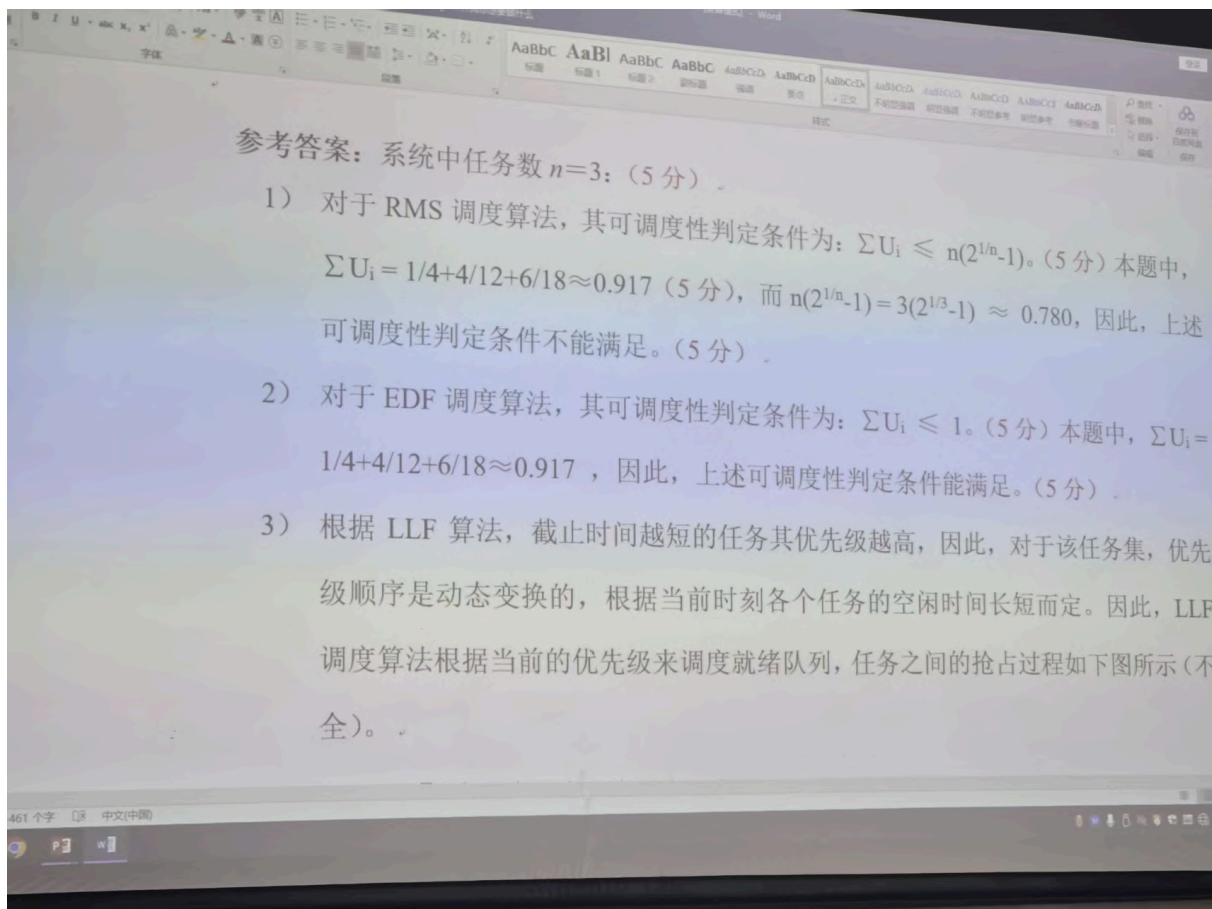
- 此时 T3 正在运行 (还剩最后 1 个单位没跑完)。
- T2 的第二个周期到达了。开始比拼:
- **算 T3:** 绝对死线 23, 当前 14, 剩余执行 = 1。
 $L_3 = 23 - 14 - 1 = 8$ 。
- **算 T2:** 绝对死线 = 26 (14+12), 当前 14, 需要执行 4。
 $L_2 = 26 - 14 - 4 = 8$ 。
- **比较:** $L_2(8) == L_3(8)$ 。它们的紧迫程度完全一样!
- **动作:** 在实际的操作系统中, 为了避免来回切换浪费性能, 遇到松弛度相同的“平局”, 通常会**让正在运行的任务继续跑完**。所以这里不抢占, 让 T3 跑到时刻 15 结束, 然后 T2 接着上。

分析方法总结

当你自己做这类题目时, 核心步骤就是:

1. **盯紧“到达时刻”和“完成时刻”:** 只有在有新任务来, 或者旧任务跑完的时候, 才需要重新计算并比较大家的松弛度。
2. **更新两个关键变量:** 在计算前, 一定要确认当前时间是多少, 以及排队任务的“剩余执行时间”还剩多少。
3. **遵循铁律:** 谁算出来的 L 越小, 就把 CPU 给谁。

只要耐心地像上面这样在一个个关键时间节点上套用公式, 就能把 0 到 44 的整个过程精确地推演出来了。你可以试着用这个方法, 自己在本子上算一下 时刻 26 发生的抢占, 看看是不是能对得上!



这两道题考察的是嵌入式实时操作系统（如 aCoral 或 uC/OS-II）中非常核心的两个机制：**任务延时管理的差分链表**以及**内存管理的伙伴系统（Buddy System）**。

下面为你详细梳理两道题的解题过程。

第 1 题：差分链表管理

核心概念：差分链表（Delta List）按任务等待时间的绝对先后顺序排列，但每个节点保存的是**距离前一个节点超时时间的差值（Delta）**。这样，系统每次发生时钟中断（tick）时，只需要将链表头结点的等待时间减 1 即可。

（1）画出由五个任务形成的初始差分链表

首先，将 T1~T5 按照等待的绝对时间从小到大排序：

- T2: 3
- T3: 6
- T1: 8
- T4: 12
- T5: 16

接着，计算每个任务与其前一个任务的时间差值（Delta）：

- T2 的差值 = $3 - 0 = 3$
- T3 的差值 = $6 - 3 = 3$
- T1 的差值 = $8 - 6 = 2$
- T4 的差值 = $12 - 8 = 4$
- T5 的差值 = $16 - 12 = 4$

形成的差分链表如下：

```
Head -> T2(3) -> T3(3) -> T1(2) -> T4(4) -> T5(4) -> NULL
```

（2）经过 7 个 tick 后，T6、T7 加入时的差分链表

经过 7 个 tick 后，我们需要先看看原来链表中剩下的任务还有多久到期：

- T2 (绝对时间 3)：已经在第 3 个 tick 执行并移出。
- T3 (绝对时间 6)：已经在第 6 个 tick 执行并移出。
- 原来绝对时间为 8 的 T1，剩余等待时间为 $8 - 7 = 1$ 。
- 原来绝对时间为 12 的 T4，剩余等待时间为 $12 - 7 = 5$ 。

- 原来绝对时间为 16 的 **T5**，剩余等待时间为 $16 - 7 = 9$ 。

此时，新任务 T6、T7 进入等待，等待时间（即剩余绝对时间）分别为 4 和 7。

将当前所有处于等待状态的任务按**剩余等待时间**重新排序：

- **T1**: 1
- **T6**: 4
- **T4**: 5
- **T7**: 7
- **T5**: 9

再次计算它们之间的差值（Delta）：

- T1 的差值 = 1
- T6 的差值 = $4 - 1 = 3$
- T4 的差值 = $5 - 4 = 1$
- T7 的差值 = $7 - 5 = 2$
- T5 的差值 = $9 - 7 = 2$

该时刻最终的差分链表如下：

```
Head -> T1(1) -> T6(3) -> T4(1) -> T7(2) -> T5(2) -> NULL
```

第 2 题：aCoral 改进伙伴系统算法回收内存

核心概念：伙伴算法在回收内存块时，会检查其相邻的“伙伴”块是否也是空闲的。如果空闲，则将它们合并成一个上一层（容量大一倍）的内存块，并递归向上检查，直到遇到被占用的伙伴块或达到最高层。

回收第 0 层第 6 块的完整过程：

1. 释放第 0 层块 6：

首先释放第 0 层的块 6。根据伙伴算法，它的伙伴是同在第 0 层的**块 7**。观察图片可知，第 0 层块 7 目前是白色（空闲）的。

2. 第一次合并（生成第 1 层块）：

由于块 6 和块 7 都空闲，它们合并成上一层的块，即**第 1 层的块 3**。此时，第 1 层块 3 变为空闲状态。

3. 检查第 1 层伙伴：

现在第 1 层块 3 空闲，检查它的伙伴——**第 1 层块 2**。观察图片可知，第 1 层块 2 也是白色（空闲）的。

4. 第二次合并（生成第 2 层块）：

将第 1 层的块 2 和块 3 合并，形成上一层的块，即**第 2 层的块 1**。此时，第 2 层块 1 变为空闲状态。

5. 检查第 2 层伙伴并终止：

现在第 2 层块 1 空闲，检查它的伙伴——**第 2 层块 0**。观察图片可知，第 2 层块 0 是灰色（已占用）的。由于伙伴被占用，**合并过程终止**。

6. 完成回收：

系统最终将合并后的**第 2 层块 1**挂入第 2 层的空闲内存块链表中。

最后的内存分配结果图描述：

经过上述回收合并后，底层的零碎内存块被组合成了更大的连续内存块。变化后的对应层级状态如下（着重看合并的区域）：

- **第 4 层**：块 0 (已占用)
- **第 3 层**：块 0 (已占用) | 块 1 (已占用)
- **第 2 层**：块 0 (已占用) | **块 1 (空闲)** | 块 2 (已占用) | 块 3 (空闲)
- **第 1 层**：块 0 (空闲) | 块 1 (已占用) | (原块 2、块 3 已合并至上层) | 块 4 (已占用) | 块 5 (空闲)
- **第 0 层**：(...省略原样部分) | (原块 6、块 7 已合并至上层) | 块 8 (已占用) | 块 9 (空闲) | ...

通过这种层层递进的合并，伙伴系统有效防止了内存碎片的产生，这也是底层 RTOS 内核设计的精妙之处。

《嵌入式操作系统》期中随堂考试

(开卷, 参考答案)

考试日期: 2025-3-28 考试时间: 90 分钟

1、列出至少两种具有自主知识产权的主流嵌入式处理器。选择一种, 介绍其特点、行业应用等。(8 分) CM2。

两种自主知识产权的嵌入式处理器: (2 分)

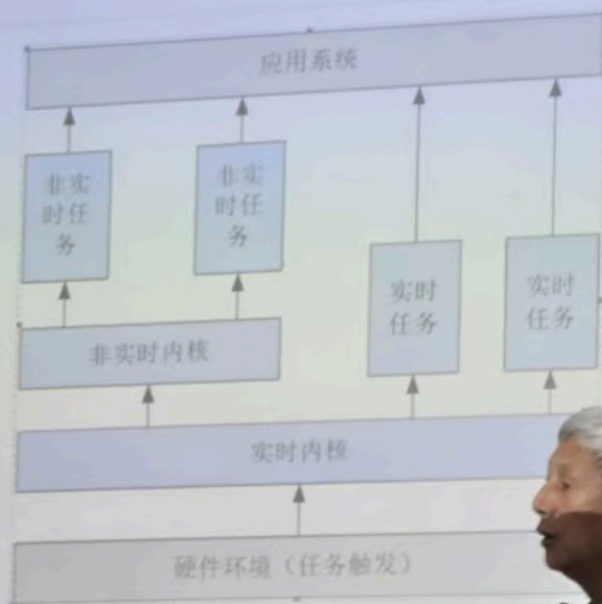
- 华为海思: HI35xx 系列、腾锐 D3000M、...

其中, HI35xx 介绍: 其特点、行业应用、等等。(6 分)

2、列出至少四种具有自主知识产权的主流嵌入式操作系统。选择一种, 进一步介绍其特点、行业应用等。(8 分) CM5。

3、结合图示，简单介绍多核嵌入式操作系统体系结构的基本思想。（10 分）

CM3



必须有体现多核的图)

内容介绍：核心、各元素简...。（6分）

5. 简单介绍嵌入式软件运行过程中的系统初始化。(8分)

系统软件运行必需的初始化工作，如根据系统配置

初始化数据空间

初始化系统所需的接口和外设

需要按特定顺序进行 (4分)

首先完成内核的初始化

完成网络、文件系统等初始化

最后完成中间件等的初始化工作

6. 什么是调度点？嵌入式实时操作系统中，有哪些调度点？对调度点“周期任务开始”，简要描述其调度处理过程。(24分) CM6

调度点 Scheduling Point，调用调度程序的具体位置 (4分)

主要的调度点 (12分，至少六种)

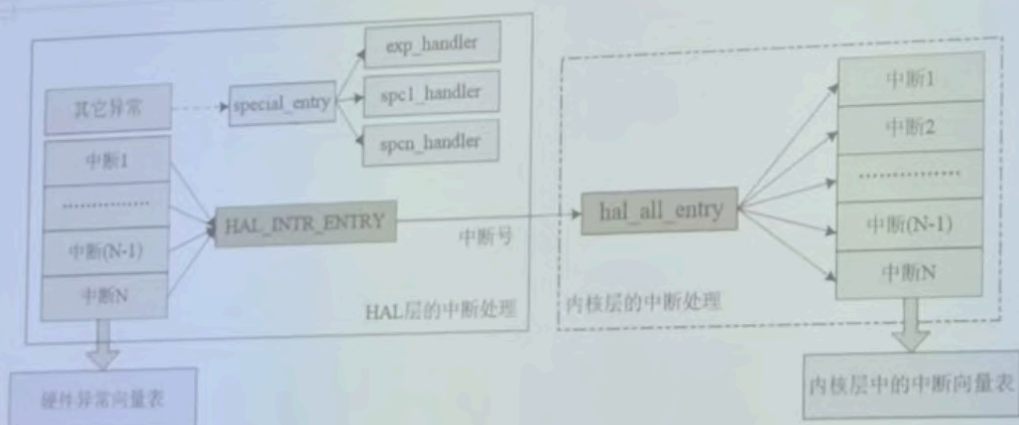
中断服务结束时

任务因等待资源而被阻塞(挂起)时

aCoral 处理流程(按下图过程介绍): (8分)

实时中断: ...

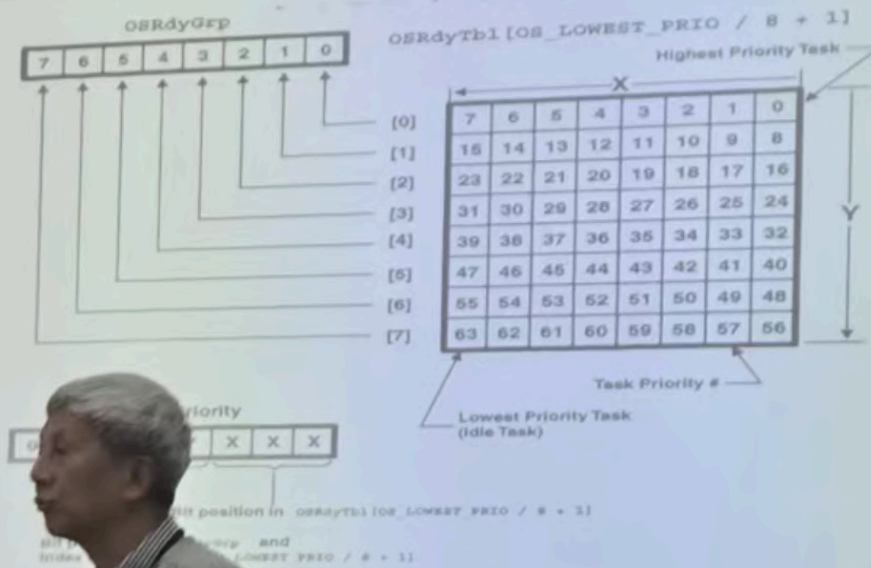
专家中断、普通中断:



如何设计其优先级位图数组？分析使用 ucOSII 的优先级位图方法，找出取最高优先级任务的基本思路。（20 分） CM6。

1) 构造 2×32 优先级位图数组 acoral prio array。（8 分）。

2) 找最高优先级任务：（12 分）。



简要介绍。

6. 什么是调度点？嵌入式实时操作系统中，有哪些调度点？对调度点“周期任务开始”，简要描述其调度处理过程。（24分） CM6。

调度点 Scheduling Point，调用调度程序的具体位置（4分）。

主要的调度点（12分，至少六种）。

- 中断服务结束时。
- 任务因等待资源而被阻塞（挂起）时。
- 周期开始或周期结束时。
- 高优先级任务达到时。
- 时间片结束时。
- 关调度等状态打开时。
- 当前任务执行结束时。

周期任务开始：调度处理过程简要介绍（可以画图；优）（8分）。

- 定时器：周期到达。
- 将所有周期到达的任务，挂就绪队列。
- 重新调度：1) 找出当前最高优先级任务。
- 2) 若是当前执行任务：继续。

8. 中断处理由哪几个阶段构成？请介绍 aCoral 的中断处理流程。（12分） CM7。

主要阶段（最好用 ucOS 模板）：（4分）。

- 中断请求。
- 现场保护。
- 执行中断服务。
- 恢复现场。
- 中断退出。

aCoral 处理流程（按下图过程介绍）：（8分）。

实时中断：—

专家中断、普通中断：—

4、说明利用 AI 工具辅助编写驱动程序的基本思路。（10 分）CM4。

核心原则：规范化（2 分）。

AI 生成的代码质量高度依赖于输入指令的清晰度。

模糊的提示（如编写串口驱动程序）会导致不可靠的代码。

明确硬件与环境：AI 不能替代对硬件的深入理解（4 分）。

芯片架构：如 Hi3559ARFCV100、RV64G（RISC-V 64 位）。

硬件模块：指定的硬件，如 TCU（张量计算单元）。

内存映射：关键寄存器的基地址和偏移量。

软件环境：指定软件环境，如 HarmonyOS 的驱动程序。

定义功能与非功能需求：参考软件工程需求分析（4 分）。

功能需求：如初始化 TCU 模块，配置其支持 INT8 精度计算。

非功能需求：如初始化延迟必须小于 10 微秒。

提供上下文与约束：帮助 AI 理解背景，如类似硬件的现有代码片断。

备注：具体设备（如串口、LED 等）可自选。

